AI Performance Engineering 2026 — Admission Assignment

View requirements, submit your solution, and track status

Task Description | Submit Solution | Check Status

Overview

In this task, you will build a simple API service that takes a GitHub repository URL and returns a human-readable summary of the project: what it does, what technologies are used, and how it's structured.

This task evaluates your ability to work with external APIs, process code repositories, and use LLMs effectively.

Technical Requirements

- **Language:** Python 3.10+
- **Web framework:** FastAPI or Flask (your choice)
- **LLM:** Nebius Token Factory API or an alternative LLM provider
- **You choose** which LLM model to use from the available models on Nebius Token Factory or an alternative LLM provider

---

Nebius Token Factory

Sign up at Nebius Token Factory to get access to LLM models.

Once signed up, you will get $1 in free credits. You will need to fill in billing detailes to get that, but you don't have to top up your account. This is mandatory and safe process.

This is more than enough to complete the task. Refer to the Nebius Token Factory documentation for available models and API usage.

As an alternative to Nebius Token Factory, you can use other LLM API provider like OpenAI, Anthropic, and Gemini. Just note that we can't provide you with free credits in that case.

Endpoint

`POST /summarize`

Accepts a GitHub repository URL, fetches the repository contents, and returns a summary generated by an LLM.

Request body:

```
{
  "github_url": "https://github.com/psf/requests"
}
```

Field	Type	Required	Description
`github_url`	string	yes	URL of a public GitHub repository

Response:

```
{
  "summary": "**Requests** is a popular Python library for making HTTP requests...",
  "technologies": ["Python", "urllib3", "certifi"],
  "structure": "The project follows a standard Python package layout with the main source code in `sr
}
```

Field	Type	Description
`summary`	string	A human-readable description of what the project does
`technologies`	string[]	List of main technologies, languages, and frameworks used
`structure`	string	Brief description of the project structure

On error, return an appropriate HTTP status code and:

```
{
  "status": "error",
  "message": "Description of what went wrong"
}
```

Key Challenges

The interesting part of this task is **how you handle the repository contents before sending them to the LLM**:

- Repositories can be large — you can't just send everything to the LLM
- You need to decide which files are important and which can be ignored (e.g., binary files, lock files, `node_modules/` , etc.)
- The LLM has a limited context window — you need a strategy for fitting the most relevant information
- Think about what gives the LLM the best understanding of a project: README? Directory tree? Key source files? Config files?

There is no single correct approach — we want to see how you think about this problem.

How Your Code Should Run

We will evaluate your solution by following the instructions in your README. Please make sure that:

1. Your README contains **step-by-step instructions** to install dependencies and start the server
2. After following your instructions, the server starts and exposes the `POST /summarize` endpoint
3. We can test it by sending a request like:

```
curl -X POST http://localhost:8000/summarize \
  -H "Content-Type: application/json" \
  -d '{"github_url": "https://github.com/psf/requests"}'
```

4. LLM API key is configured via the `NEBIUS_API_KEY` **environment variable** (we will provide our own key when testing). Do not hardcode API keys. In case you are using an alternative LLM provider, you can use corresponding environment variable name, for example `OPENAI_API_KEY` , `ANTHROPIC_API_KEY` etc.

That's it — if we can follow your README and get a working response, you're good.

What to Submit

1. **Working source code** for the API service
2. `requirements.txt` (or equivalent, e.g., `pyproject.toml`) with all dependencies
3. `README.md` with:
   - Step-by-step setup and run instructions (assume a clean machine with Python installed)
   - Which model you chose and why (1–2 sentences is fine)
   - Your approach to handling repository contents (what you include, what you skip, and why)

Upload your solution as a **zip archive** at Submit tab.

Evaluation Criteria

Your submission is scored on a **10-point scale** across the criteria below. You do not need a perfect score to pass — we are looking for a solid, working solution with thoughtful decisions.

Blocking criteria

These must be met for the submission to be evaluated. If any of these fail, the submission receives 0 points:

- The server starts following the instructions in the README
- The `POST /summarize` endpoint exists and accepts the specified request format
- The endpoint returns a response (not an error) for a valid public GitHub repository
- The Nebius Token Factory (or an alternative provider) API is used for LLM calls

Scoring

Criteria	Points	What we look for
Functionality	20	The endpoint returns a meaningful, accurate summary for different repositories. Response matches the specified format (`summary` , `technologies` , `structure`).
Repo processing	20	Files are filtered sensibly (binary files, lock files, etc. are skipped). There is a clear strategy for selecting the most informative files.
Context management	20	The solution handles large repositories without crashing or sending too much to the LLM. There is some strategy for staying within context limits (truncation, prioritization, summarization, etc.).
Prompt engineering	10	The prompt(s) to the LLM are clear and produce structured, useful output.
Code quality & error handling	20	Code is readable and reasonably organized. Edge cases are handled (invalid URL, private repo, empty repo, network errors). API keys are not hardcoded.

Criteria	Points	What we look for
Documentation	10	README includes working setup instructions and a brief explanation of design decisions.

Total: 100 points

Don't overthink it. A clean, working solution that handles the main scenarios well is what we're looking for. If you have time and want to go further — great, but it's not required.

Good luck!

Need help? Contact support.academy@nebius.com.